

Code review — academy

Runde de review pe mini-ERP-ul de academie, cea mai nouă sus.

Proiect: mini-ERP de academie, C# consolă, fără framework. Regula fișierului: runda nouă se adaugă **sus**. Rundele vechi rămân dedesubt, condensate.

Runda 1 — 2026-09-08 — sha 79abec3

Scop: verificat commit-ul 79abec3 „fixes T0” — adică toate cele cinci task-uri T0.0–T0.4 din TASKS.md . Metodă: build + rulat aplicația pe date reale, plus un mic banc de test separat pentru codul la care meniul nu ajunge. Codul tău nu a fost modificat.

Poarta de compilare

```
Build succeeded.  
    75 Warning(s)  
    0 Error(s)
```

Trece. Prima dată în trei commit-uri consecutive când primesc cod care compilează. Ține regula.

T0 — verdict pe task-uri

Task	Stare	Cum am verificat
T0.0 — nu compilează	✅ închis	<code>dotnet build</code> → 0 erori
T0.1 — „Vezi cursurile” crapă	✅ închis	logat <code>Scarlat / Stefan</code> → <code>2</code> → <code>1</code> → afișează <i>Analiza</i> și <i>Algoritmica</i> , fără excepție
T0.2 — modificarea orfanizează cartea	✅ închis	<code>1</code> → <code>1</code> → <code>3</code> (<code>Cartea1</code> → <code>Cartea9</code>) → <code>1</code> : cartea rămâne a ta, cu numele nou și data inițială <code>2025-12-03</code> neschimbată
T0.3 — <code>createdAt</code> ignorat	✅ închis	<code>new Enrolment(id, id, 2020-03-15)</code> → <code>CreatedAt = 2020-03-15</code>
T0.4 — <code>CreateUser</code> face mereu un <code>User</code> gol	✅ închis	<code>CreateUser(TeacherCreateRequest(...))</code> → tip real <code>Teacher</code> , <code>salary=5000</code> <code>workHours=30 password='parola123'</code>

5/5. Toate reparate pe fond, nu doar pe suprafață.

Două lucruri făcute bine, pe care nu ți le ceruse nimeni: ai curățat `using` -urile nefolosite din 8 fișiere, și la T0.1 n-ai peticit metoda existentă, ci ai scris `GetCourseIdListByStudentId` — pasul care lipsea din lanț. Traducerea „înrolare → curs” e acum o metodă cu nume, nu o presupunere ascunsă.

Critice

B1 — Enter pe gol înseamnă „prima carte” / „primul curs”

`Courses/Services/CourseService.cs:45` · `Books/Services/BookService.cs:32`

CUM E ACUM: logat `Popescu / Daniel` (are doar *Geometrie* și *Cartea4*):

```
2 → 2 → [Enter, fără să scriu nimic]
Ati fost inscris cu succes la curs!
2 → 1
    nume: Geometrie, departament: Matematica
    nume: Analiza, departament: Matematica    <-- nu am cerut asta
```

Și varianta care distruge date:

```
1 → 4 → [Enter, fără să scriu nimic]
Carte stearsa cu succes
1 → 1
    Nu ai nicio carte
```

DE CE: `FindByName` și `GetBook` caută cu `Contains`, nu cu egalitate. `Contains` răspunde la întrebarea „numele conține fragmentul ăsta?”, iar șirul gol e conținut în absolut orice șir — `"Analiza".Contains("")` e `true` prin definiție, la fel ca la orice altceva. Deci prima iterație a buclei se oprește și îți întoarce **primul element din listă**, oricare ar fi el.

Nu e nevoie nici măcar de Enter gol ca să te muște: `"a"` se potrivește tot cu *Analiza*, deși tu voiai *Algebra*. Bucla se oprește la primul care se potrivește, nu la cel mai bun. Iar în cazul ștergerii, între „am tastat greșit” și „am pierdut o linie din bază” nu mai există niciun pas.

FIX: căutarea după nume e egalitate, nu potrivire de fragment — `c.Name == name` (eventual `string.Equals(c.Name, name, StringComparison.OrdinalIgnoreCase)` dacă vrei să ierți majusculele). Separat, respinge inputul gol în `View` înainte de a intra în serviciu, și cere o confirmare `da/nu` înainte de `DeleteBook`.

Aceeași regulă e încălcată în ambele locuri de mai sus. Repară-le pe amândouă în același commit.

B2 — T0.1 a reparat linia, nu regula: lista de cursuri încă poate conține `null`

`Courses/Services/CourseService.cs:29–39` (linia 35) · consumat în `ViewStudent.cs:126–133`

CUM E ACUM: metoda are acum numele corect și primește ID-uri de curs, deci scenariul din T0.1 merge. Dar corpul buclei a rămas neatins:

```
foreach (Guid id in coursesId)
{
    studentCourses.Add(FindById(id)); // FindById poate întoarce null
}
```

Rulat pe un ID de curs care nu există în `courses.txt` :

```
GetCourseListByCourseId cu un id inexistent -> Count=1, primul element este null? True
afisarea lui -> NullReferenceException
```

Onest: asta am reprodus-o pe un banc de test, nu din meniu. Astăzi niciun ecran nu șterge cursuri, deci lanțul nu se rupe în practică. Devine reproductibil din UI în clipa în care apare `ViewAdmin` cu ștergere de cursuri — sau dacă cineva editează cu mâna `enrolments.txt`.

DE CE: T0.1 avea două cauze suprapuse, și tu ai închis-o pe cea vizibilă. Cauza 1: trimiteai ID-uri de înrolare unde se așteptau ID-uri de curs — reparată. Cauza 2: `FindById` are voie să întoarcă `null`, iar apelantul îl adaugă în listă fără să se uite la el. Cauza 2 a produs crash-ul original la fel de mult ca și cauza 1: lista *nu* era goală, era plină de `null`-uri, de-aia `courses.Count > 0` era adevărat și bucla intra în `c.Name` pe un `null`.

Cu alte cuvinte: `Count > 0` verifică **câte** elemente sunt, nu **ce** sunt. Cât timp o metodă poate strecura `null` într-o listă, orice verificare de lungime făcută după ea minte.

Ăsta e tiparul pe care ți-l semnez a doua oară: repari apariția semnalată și lași regula în picioare. La runda următoare, după fiecare fix, întreabă-te „unde mai poate ieși `null` din chestia asta?” — nu doar „mai crapă scenariul din task?”.

FIX: ori sari peste ce nu găsești, ori refuză explicit — dar nu adăuga `null` în listă:

```
Course c = FindById(id);
if (c != null) studentCourses.Add(c);
```

B3 — bomba de la T1 e acum armată: un profesor salvat nu mai poate fi citit

`Users/Models/Teacher.cs:82-96` (`ToText`) vs `Users/Models/Teacher.cs:28-33` (constructorul din text)

CUM E ACUM: creat un profesor, apoi un student, apoi `Save()`, apoi repornit:

```
--- users.txt scris; ultimele 2 linii: ---  
TEACHER,d0089b4f-...,Ion,Popescu,ion@gmail.com,40,5000,30  
STUDENT,59ddf471-...,Maria,Ionescu,maria@gmail.com,20  
--- repornire (recitare): ---  
CRASH: IndexOutOfRangeException: Index was outside the bounds of the array.
```

Și varianta „nu crapă, dar e mai rea” — profesorul scris ca **ultima** linie din fișier:

```
TEACHER,c23ea4f8-...,Ion,Popescu,ion@gmail.com,40,5000,30,
```

Se recitește fără excepție, dar cu `Password = ""`. Profesorul rămâne în bază și nu se mai poate loga niciodată.

DE CE: `ToText` scrie 8 câmpuri și uită parola. Constructorul `Teacher(string text)` citește `cuv[8]` — al nouălea. Scrierea și citirea sunt două funcții aflate în capete opuse ale clasei și au divergat fără ca nimic să le lege: compilatorul nu compară o concatenare de string-uri cu un `Split`, așa că nimeni nu-ți spune nimic până la rulare.

Cele două comportamente diferite vin din ramura `if (cnt + 1 == size)`: pe ultima linie `ToText` pune o virgulă în plus la final, ceea ce produce accidental un `cuv[8]` gol; pe orice altă linie pune `\n` fără virgulă, deci `cuv` are doar 8 elemente și `cuv[8]` iese din tablou. Același bug se manifestă în două feluri în funcție de **poziția** rândului în fișier — de-asta e greu de prins prin citire.

Ce s-a schimbat față de ieri: până la `79abec3`, `CreateUser` nu producea niciodată un `Teacher` real, deci linia asta nu se scria niciodată. **Reparând T0.4 ai armat T1.** Nu e o greșeală — e exact ordinea pe care ți-am cerut-o. Doar că acum e adevărat, nu teoretic.

`Admin` face corect același lucru (scrie 8 câmpuri, citește `cuv[6]` și `cuv[7]`) — deci nu e o regulă pe care n-o știi, e o clasă în care ai scăpat-o. Compară `Teacher.ToText` cu `Admin.ToText` una lângă alta.

FIX: asta e **T1** din `TASKS.md` și se rezolvă prin construcție, nu prin peticire: `ITextMapper<T>` cu `ToText` și `FromText` în aceeași clasă, una sub alta, ca să nu mai poată diverge. Ai creat deja `Users/Models/ITextMapper.cs` cu semnătura corectă — următorul pas e să-l implementezi pentru `Teacher` și să muți `Save / Read` pe el. **Până atunci nu chema `Save()` nicăieri** (vezi B4 din runda de analiză: încă nu e chemat, și e bine așa).

B4 — orice tastă greșită în meniu omoară aplicația

ViewStudent.cs:34 · :63 · :87 · :106

CUM E ACUM: logat ca student, apăsând `x` în meniul principal:

```
Unhandled exception. System.FormatException: The input string 'x' was not in a correct format
at System.Int32.Parse(String s)
at ViewStudent.Viewer() in ViewStudent.cs:line 34
```

DE CE: `Int32.Parse` are un singur mod de a răspunde la un text care nu e număr: aruncă. Excepția urcă prin `Viewer` → `Logger` → `Main`. Iar `Main` prinde `catch (ArgumentException)`, care nu acoperă `FormatException` — sunt două clase-surori sub `SystemException`, niciuna nu derivă din cealaltă. Deci nimic nu o oprește și procesul moare.

Ai deja ramura `default: InputGresit();` în toate cele patru `switch`-uri — dovada că te-ai gândit la inputul greșit. Doar că ea prinde „ai tastat `7`”, nu „ai tastat `x`”: la `7` conversia reușește și ajungi în `switch`, la `x` mori înainte.

FIX: `Int32.TryParse`, care întoarce `bool` în loc să arunce:

```
if (!Int32.TryParse(Console.ReadLine(), out tasta))
{
    InputGresit();
    continue;
}
```

Același bug în toate cele patru meniuri (`Viewer`, `Carti`, `Cursuri`, `Statistici`). Repară-le pe toate patru, nu doar pe cel din stack trace.

🟡 Importante

- M1 — Create din modele a devenit cod mort, iar logica lui e acum copiată în serviciu.**
`UserService.cs:82-117` copiază câmp cu câmp din request în model, deci `Student.Create` (`Student.cs:33-36`), `Teacher.Create` (`Teacher.cs:98-105`) și `Admin.Create` (`Admin.cs:72-78`) nu se mai execută niciodată. Atenție la asimetrie: `Update` a rămas polimorfic și viu (`UserService.cs:151`), `Create` nu. Acum ai două copii ale aceleiași reguli „request →

model", care pot diverge exact cum au divergat `ToText` și `FromText` la B3. Se rezolvă definitiv la **T3 (Factory Method)** — până atunci, măcar cheamă `newT.Create(t)` din serviciu în loc să repeți atribuirile.

- **M2 — ✓ ÎNCHIS de T0.5 (Book).** `BookUpdateRequest` e acum `record`
`BookUpdateRequest(string BookName)` — data nu mai poate fi trimisă, deci nu mai poate fi ignorată. Textul original, pentru context: `BookUpdateRequest` cerea o dată pe care serviciul o ignora în tăcere. Ca să închizi T0.2 ai scos `book.CreatedAt = request.CreatedAt;` din `BookService.UpdateBook ( :135–136 )`. Rezultatul e corect, dar constructorul `BookUpdateRequest(studentId, bookName, createdAt)` (`Books/Dtos/BookUpdateRequest.cs:12`) încă cere `DateTime.Now`, iar `ViewStudent.cs:194` i-l dă degeaba. E **exact bug-ul pe care tocmai l-ai reparat la T0.3**, pe dos: acolo constructorul primea `createdAt` și nu-l folosea. Fix corect: scoate parametrul din DTO, ca să nu mai poată fi trimis. Cauza de dedesubt e `BookUpdateRequest : Book` — câmpul n-a fost decis de nimeni, a venit gratis odată cu moștenirea; **vezi T0.5 în TASKS.md**, unde e tratată împreună cu celelalte 5 request-uri care moștenesc entitatea.
- **M3 — cod mort după T0.1:** `EnrolmentService.GetEnrolmentIdByStudentId ( :26–40 )` nu mai e chemată de nimeni. Șterge-o; altfel la runda următoare cineva (tu) o va chema din greșeală în locul celei corecte.
- **M4 — modificarea unei cărți nu confirmă nimic.** `ViewStudent.cs:195` — `response` e atribuit și nefolosit, deci ecranul tace. `AdaugareCarte` afișează mesaj de succes; fă la fel aici.
- **M5 — User newUser = new(); (UserService.cs:76) construiește un obiect aruncat imediat** în toate cele trei ramuri utile. Declar-o fără inițializare, sau întoarce direct din fiecare ramură.
- **M6 — InscriereCurs ocolește serviciul:** `ViewStudent.cs:238` adaugă direct în `enrolmentService.Enrolments`, lista internă expusă prin proprietate. Toate validările din `CreateEnrolment` sunt sărite. Serviciul ar trebui să întoarcă o copie, nu lista lui.
- **M7 — login-ul de admin/profesor iese tăcut.** Cu parola corectă, `ViewLogIn.cs:39` și `:53` conțin doar comentariile `//viewAdmin` / `//viewTeacher`, deci metoda se termină și procesul iese cu 0. Verificat: `Grozavu / Alex / gicuEgrozav` → nimic. Până scrii ecranele, pune măcar un `Console.WriteLine("Ecran indisponibil")`.
- **M8 — serviciile se instanțiază de două ori.** `ViewLogIn.cs:10` face `new UserService()`, `ViewStudent.cs:14–16` face alte trei. Fiecare `new` recitește fișierul și creează o a doua copie în memorie. Cu `ViewAdmin` și `ViewTeacher` vor fi trei seturi paralele care nu se văd între ele. Discuția asta e **T4**; răspunsul corect e să dai serviciile prin constructor, cum faci deja cu `ViewStudent(User user)`.

Cleanups

- **C1** — `Console.WriteLine(aux);` rămas din depanare, `ViewStudent.cs:275` — la statistici afișează `1 2 1 1 2 1` înainte de rezultat.
 - **C2** — încă nu există `.gitignore`. `bin/` și `obj/` apar ca *untracked* la primul build; e o chestiune de timp până ajung în istoric.
 - **C3** — `Data/students.txt` e mort: `grep students.txt` peste tot codul nu găsește nimic. Șterge-l, ca să nu pară a doua sursă de adevăr lângă `users.txt`.
 - **C4** — parole în clar, comise în `Data/users.txt` (`gicuEgrozav`). E un proiect de învățare, deci nu e urgent — dar află de ce se face altfel înainte de a scrie ceva real.
 - **C5** — 75 de warnings. Majoritatea sunt `CS8600` / `CS8602` / `CS8604` pe `Console.ReadLine()`, care poate întoarce `null`. Sunt aceleași locuri unde te mușcă B4. Warning-urile îți arătau deja problema.
 - **C6** — `Student.Create` și `Student.Update` (`Student.cs:33–41`) nu fac decât `base.X(request)`. Un `override` care doar cheamă `base` poate fi șters — nu adaugă nimic.
 - **C7** — ai curățat `using` -urile în 8 fișiere , dar au rămas în 30 (`Student.cs`, `Teacher.cs`, `Book.cs` și toate DTO-urile). `Ctrl+K`, `Ctrl+E` în Visual Studio le face pe tot fișierul.
-

Before / After pentru cele

#	Acum	Cum ar trebui
B1	<pre>if (c.Name.Contains(name)) if (book.StudentId == studentId && book.BookName.Contains(bookName))</pre>	<pre>if (c.Name == name) if (book.StudentId == studentId && book.BookName == bookName) + în View : if (string.IsNullOrEmpty(text)) { Console.WriteLine("Numele nu poate fi gol"); return; }</pre>
B2	<pre>foreach (Guid id in coursesId) { studentCourses.Add(FindById(id)); }</pre>	<pre>foreach (Guid id in coursesId) { Course c = FindById(id); if (c != null) studentCourses.Add(c); }</pre>
B3	<pre>ToText : ... + Salary + "," + WorkHours + "\n" Teacher(string) : Password = cuv[8];</pre>	Un singur loc care știe formatul — TeacherTextMapper : ITextMapper<Teacher> , cu ToText și FromText una sub alta. Ambele scriu/citesc 9 câmpuri, iar \n -ul îl pune Repository , nu modelul. (= T1)
B4	<pre>tasta = Int32.Parse(Console.ReadLine());</pre>	<pre>if (!Int32.TryParse(Console.ReadLine(), out taste)) { InputGresit(); continue; }</pre>

Întrebări de verificare

1. **B1.** `"Analiza".Contains("")` întoarce `true`. De ce e adevărat prin definiție, și nu un caz special pe care l-au uitat cei de la Microsoft? Pornind de la răspuns: în ce situație e `Contains` unealta potrivită pentru o căutare — și ce trebuie să fie diferit față de ecranele tale ca s-o poți folosi liniștit?
2. **B2.** La T0.1 aplicația crăpa cu `NullReferenceException` deși `courses.Count > 0` era adevărat. Explică în două fraze cum pot fi ambele adevărate în același timp. Apoi: mai există în cod alt loc unde adaugi într-o listă rezultatul unei metode care poate întoarce `null`?
3. **B3.** Un profesor salvat ca **ultima** linie din `users.txt` se recitește fără excepție, dar cu parola goală. Același profesor salvat pe **penultima** linie face aplicația să crape la pornire. Aceeași greșală — de ce două rezultate diferite? Și de ce e varianta „fără excepție” mai periculoasă decât cea care crapă?
4. **M1.** După fix-ul de la T0.4, `Teacher.Create` nu mai rulează niciodată, dar `Teacher.Update` rulează. Uită-te la `UserService.CreateUser` și la `UserService.UpdateUser` și spune care e diferența care produce asta. (Răspunsul e propoziția de la T0.4 — și e exact ușa către T3.)

Ce urmează

1. Cele patru 🚫 de mai sus — **B1 și B4 primele**, sunt cele pe care le lovește orice utilizator în primele două minute.
2. **T0.5 — DTO-urile moștenesc entitățile** (task nou în `TASKS.md`): request-urile devin `record`-uri, iar traducerea DTO ↔ entitate trece în întregime prin mappere. **Book e deja făcut în cod, ca model de urmat** — tu faci `Course` și `Enrolment`, apoi `Users`. Închide M2 prin construcție și face bug-ul de la T0.2 imposibil de scris, nu doar reparat.
3. **T1 — Strategy** (`ITextMapper<T>` + `Repository<T>`). Ai pus deja interfața; mai lipsește implementarea. DoD-ul rămâne cel din `TASKS.md`: pui un TEACHER în **mijlocul** lui `users.txt`, salvezi, repornești — trebuie să pornească. Testul ăla pică acum, l-am rulat.
4. **T2 — Observer** (`Save()` chemat automat). Nu înainte de T1: primul `Save()` peste formatul actual strică baza.

